

Distributed Stencil Performance Analysis

High Performance Computing

297SM

23/01/2026

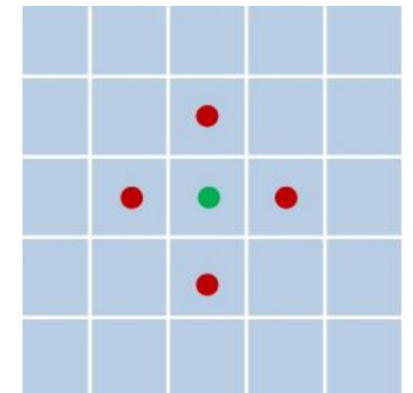


Goal of the Assignment

- Implement a parallel 5-point stencil to compute heat diffusion computation
- Parallelize C code, combining the advantage of **MPI** for distributed memory communications and **OpenMPI** directives for shared memory.
- Conduct scalability analysis in a **Leonardo - Cineca** High Performance / Productivity environment.

The 5-point stencil computation

- Stencil computations have diverse applications including the numerical solution of partial differential equations, such as the **heat** equation for thermal diffusion process.
- The continuous domain is **discretized** into a two dimensions grid where the state of each cell is updated iteratively. Using the finite difference method, a point's new value $x_{ij}(t)$ is derived from the average of its previous value $x_{ij}(t-1)$ and its four orthogonal neighbours.
- The parallelization strategy decomposes the global simulation grid into local patches, assigning one patch per MPI rank.
- This introduces data dependencies across process boundaries and necessitates **inter-process communication**.



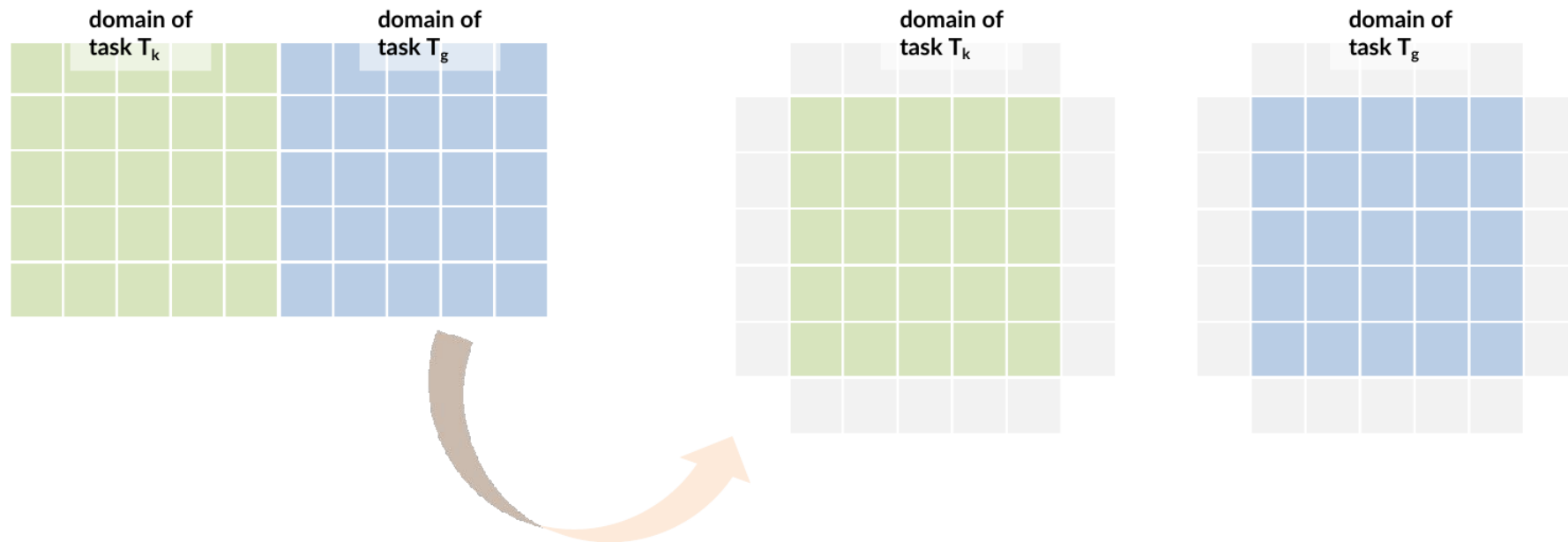
5-point

Parallelization strategy: Domain Decomposition

- Parallelization begins by partitioning the global computational grid into smaller sub-grids, with each **MPI process** managing a specific region.
- While *internal points* can be updated independently, the *boundary points* introduce dependencies on data held by neighbouring processes.
- Border points at the edge of a local domain require values from adjacent domains to complete the stencil operation. Consequently, synchronization and data exchange between processes are performed at each time step.

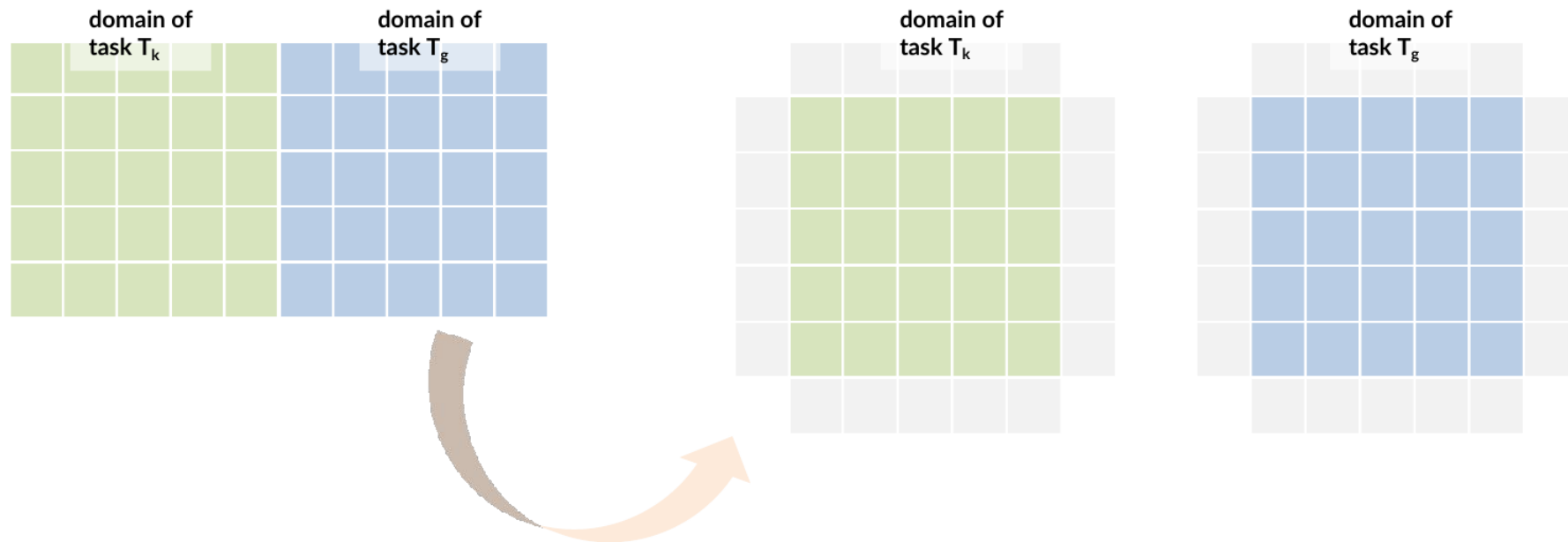
Parallelization strategy: Domain Decomposition

- A custom 2D Cartesian topology that maps MPI ranks to grid coordinates was implemented.
- **Grid Partitioning:** An automatic factorization algorithm `simple_factorization()` determines the optimal process grid dimensions to ensure load balancing and minimize the surface-to-volume ratio (square-like decomposition).



Parallelization strategy: Domain Decomposition

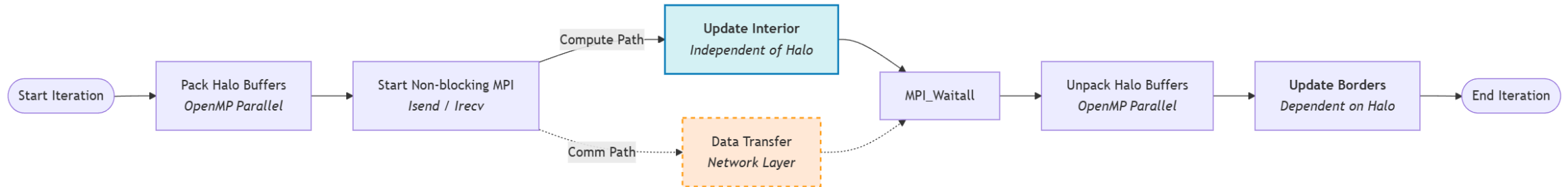
- Each process calculates its coordinates (X, Y) in the virtual grid using arithmetic operations ($X = \text{Rank} / \text{Grid}[x], Y = \text{Rank} / \text{Grid}[x]$), and determines its neighbours (N, S, E, W) through direct rank calculations. This manual approach provides explicit control over the decomposition logic and avoids the overhead of MPI topology management functions.



Halo exchange: border computation

- To enable border point computation, each MPI rank process reserves additional memory regions (*Halo zones*) surrounding its local grid patch, which will hold boundary values received from adjacent ranks.
- The halo exchange follows a coordinated protocol: each rank prepares its boundary data, transmits it to neighbouring ranks via MPI, and simultaneously receives corresponding boundary data to populate its halo regions.
- Implementation:
 - Non-blocking MPI calls (`MPI_Isend` / `MPI_Irecv`) for asynchronous communication
 - Inline implementation within the main loop (not separate functions) for minimal overhead
 - Parallel packing/unpacking using OpenMP (for East/West only) achieving 3-4× speedup
 - Self-communication optimization: If a neighbour is the same MPI rank (periodic boundaries), data is copied directly using OpenMP instead of MPI, eliminating network overhead

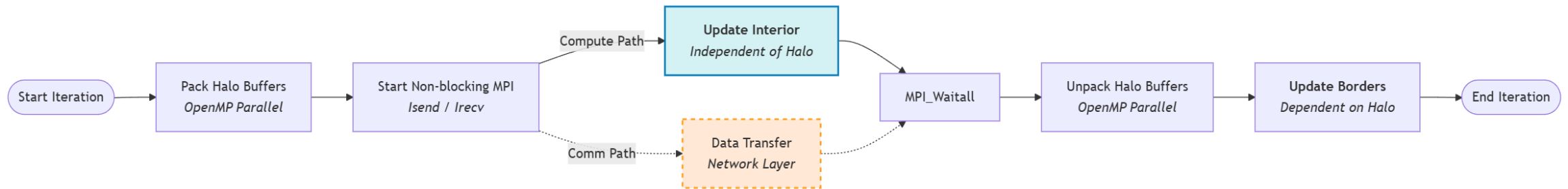
Communication-Computation Overlap



To hide the latency of **MPI communications**, a non-blocking communication strategy overlapped with computation was adopted. The halo exchange is implemented inline within the main iteration loop to minimize function call overhead and enable control over the communication pattern. The stencil update is split into two distinct kernels:

- **update_interior()** Updates the inner core of the sub-grid, which does not depend on halo data. This function is parallelized using **OpenMP** (`#pragma omp parallel for schedule(static)`) to distribute the computation across threads within each MPI process.
- **update_borders()** Updates the edges of the sub-grid, which depend on neighbour data. This function is also parallelized with **OpenMP**, using separate loops for horizontal and vertical borders to maximize cache efficiency.

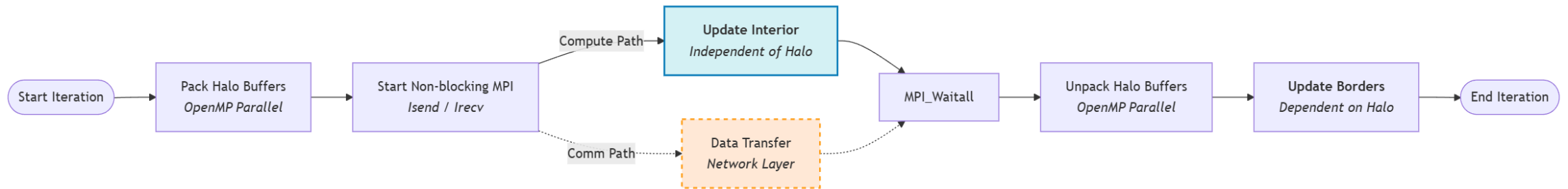
Halo Exchange Phases 1/4



Phase 1 - Halo Exchange Start (inline implementation):

- **East/West borders:** Pack non-contiguous column data into contiguous send buffers using **OpenMP** parallelization.
- **North/South borders:** Set up direct pointers to contiguous row data (no packing needed).
- **Post non-blocking receives** (`MPI_Irecv`) for all neighbours, followed by non-blocking sends (`MPI_Isend`).
- **Self-communication optimization:** If a neighbour is the same MPI rank (if periodic boundaries is active), copy data directly using OpenMP parallel loops instead of MPI calls, eliminating network overhead and reducing latency.

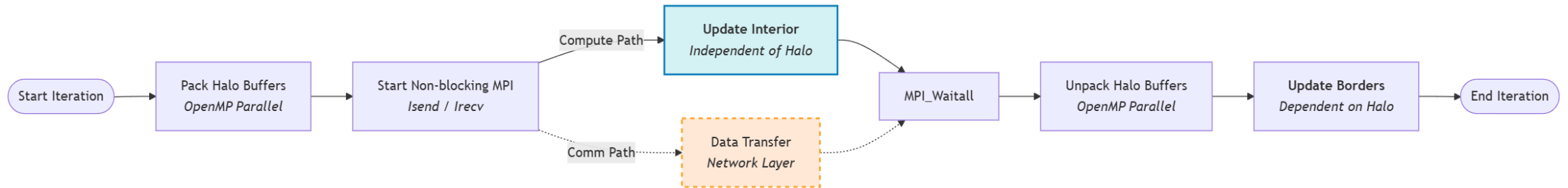
Halo Exchange Phases 2/4



Phase 2 - Computation Overlap:

- Immediately call `update_interior()` to compute interior points while MPI transfers halo data in the background. This function uses OpenMP to parallelize the computation across threads within the MPI process.

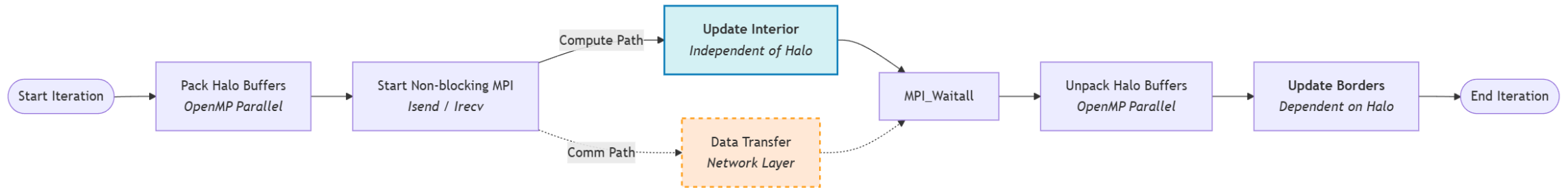
Halo Exchange Phases 3/4



Phase 3 - Halo Exchange Finish:

- Call `MPI_Waitall()` to ensure all communications are complete.
- East/West: Unpack received data from buffers into halo regions using OpenMP parallelization.
- North/South: Data is already in place via direct pointers (no unpacking needed).

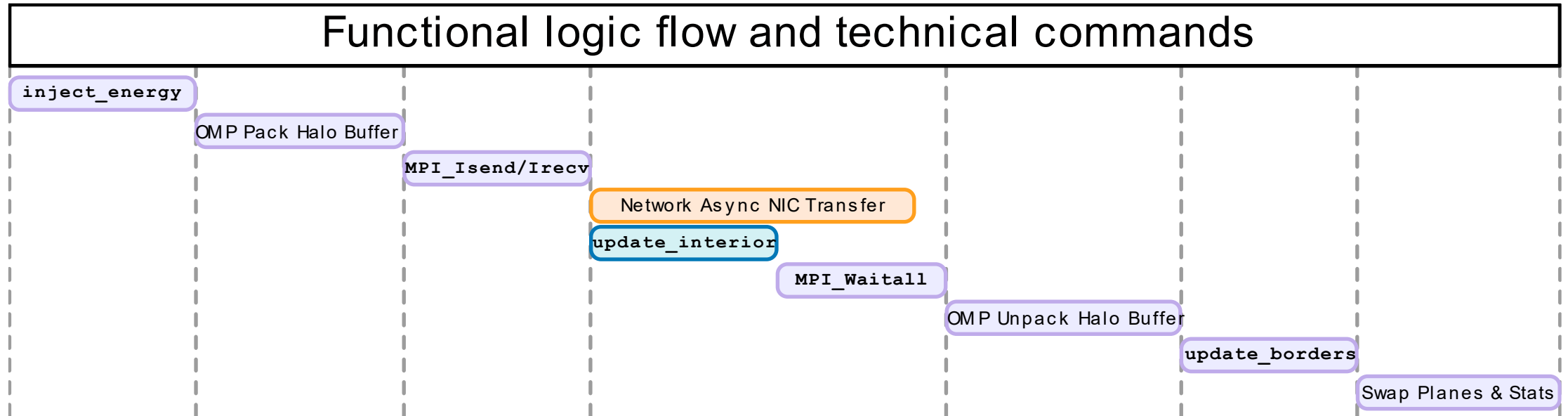
Halo Exchange Phases 4/4



Phase 4 – Border Computation:

- Call `update_borders()` to compute border points using the fresh halo data. This function also uses OpenMP parallelization, with separate loops for horizontal and vertical borders to optimize cache access patterns.

Parallel Stencil: GANTT Chart



OMP Pack / Unpack Halo Buffer:

The `#pragma omp parallel for schedule(static)` directive parallelizes the loop that copies non-contiguous column data (East/West borders) into contiguous send buffers, distributing iterations evenly across OpenMP threads.

#pragma omp parallel for schedule(static) 1/5

#pragma omp parallel for schedule(static)

- This directive is a combined construct that spawns a team of threads (`parallel`) and immediately distributes the iterations of the following loop across those threads (`for`).
- The `schedule(static)` clause dictates the partitioning strategy: the total number of loop iterations is divided into equal-sized chunks (by default, `total_iterations / num_threads`) and assigned to threads in a **round-robin** fashion at the start of the parallel region.
- Because the mapping is deterministic and fixed before execution begins, `static` scheduling introduces **near-zero runtime synchronization** overhead compared to `dynamic` or `guided` schedules. This makes it the optimal choice for loops with uniform computational intensity per iteration and **no load imbalance** (e.g., copying a rectangular memory region).
- Furthermore, when used consistently across multiple loops operating on the same data structures, it promotes **NUMA locality**: the same thread that initializes a memory page (via ***first-touch***) is likely to process it in subsequent loops, ensuring data remains in the CPU's local cache or L1/L2 cache.

#pragma omp parallel for schedule(static) 2/5

#pragma omp parallel for schedule(static)

1. Stencil Computation (Core Loops)

- `update_interior()` : Parallelizes interior grid points computation (1 pragma)
 - Loop bounds: $j \in [2, \text{ysize}-1]$, $i \in [2, \text{xsize}-1]$ (excludes borders)
 - Executes during MPI communication overlap
- `update_borders()` : Parallelizes border points computation (2 pragmas)
 - Horizontal borders (top/bottom): separate loop
 - Vertical borders (left/right): separate loop
- Requires halo data from neighbours

#pragma omp parallel for schedule(static) 3/5

#pragma omp parallel for schedule(static)

2. Halo Exchange Operations

- Pack East/West buffers: Parallel copying of non-contiguous column data into send buffers (2 pragmas)
- Self-communication: When neighbor == own rank, parallel copy replaces MPI calls (4 pragmas for all directions)
- Unpack East/West buffers: Parallel copying of received data into ghost cells (2 pragmas)

#pragma omp parallel for schedule(static) 4/5

#pragma omp parallel for schedule(static)

3. Memory Management

- First-touch NUMA allocation: Parallel initialization of grid memory for NUMA-aware placement (2 pragmas for OLD and NEW planes)
- Ensures memory pages allocated on local NUMA nodes

```
#pragma omp parallel for schedule(static)
for (unsigned int i = 0; i < frame_size; i++) {
    planes_ptr[OLD].data[i] = 0.0;
}

planes_ptr[NEW].data = (double*)malloc( frame_size * sizeof(double) );
if ( planes_ptr[NEW].data == NULL )
    return -1;

// First-touch for NEW plane as well
#pragma omp parallel for schedule(static)
for (unsigned int i = 0; i < frame_size; i++) {
    planes_ptr[NEW].data[i] = 0.0;
}

// =====
// FIRST-TOUCH ALLOCATION WITH OPENMP (NUMA OPTIMIZATION)
// =====
// #pragma omp parallel for schedule(static)
// - Parallelizes the initialization loop across threads
// - Each thread initializes a portion of the array
//
// WHY FIRST-TOUCH?
// - On NUMA (Non-Uniform Memory Access) systems, memory pages
//   are physically allocated on the NUMA node where they are
//   FIRST TOUCHED (written), not where malloc is called
// - By having each thread touch its portion of the array,
//   we ensure memory is distributed across NUMA nodes
// - This matches the data access pattern in the computation:
//   each thread will work on the same data it initialized
//
```

#pragma omp parallel for schedule(static) 5/5

#pragma omp parallel for schedule(static)

4. Reduction Operations

- `get_total_energy()` : Parallel sum using reduction(+:totenergy) (1 pragma)
- Thread-safe global energy calculation across all grid points

```
#pragma omp parallel for schedule(static) reduction(+:totenergy)
for ( int j = 1; j <= ysize; j++ )
    for ( int i = 1; i <= xsize; i++ )
        totenergy += data[ IDX(i, j) ];

#undef IDX

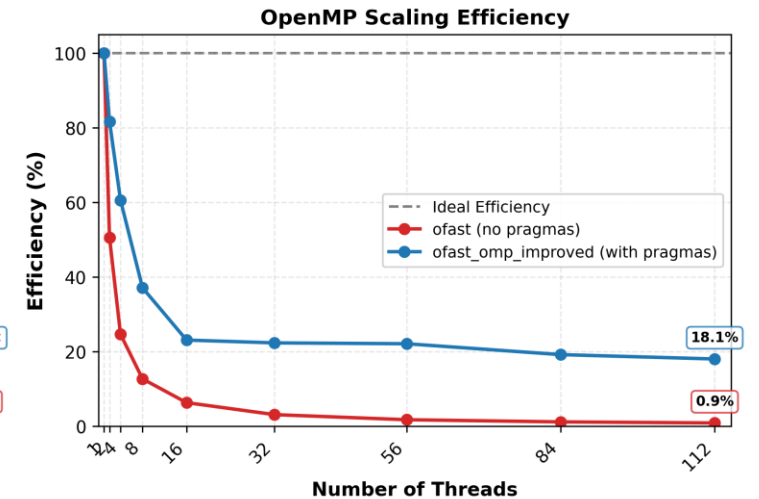
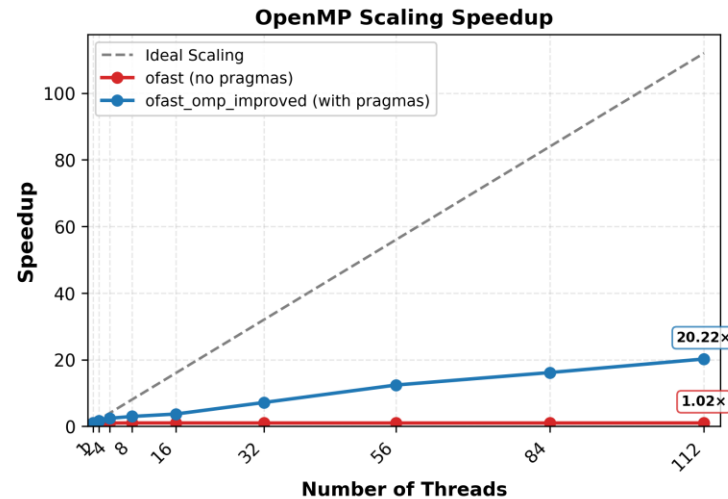
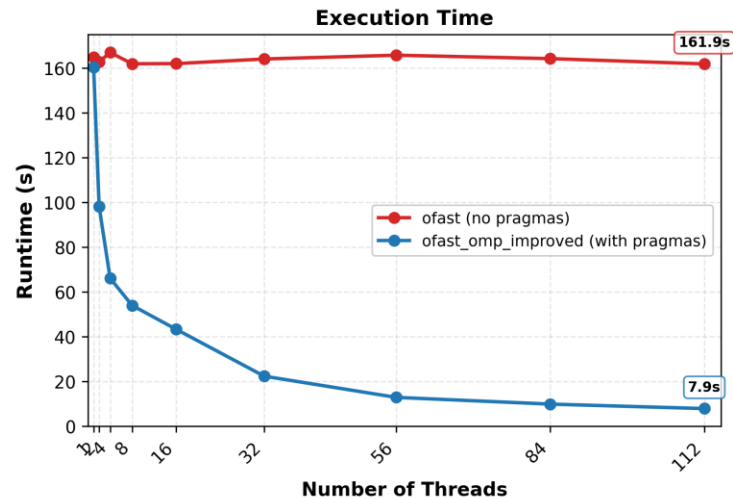
*energy = (double)totenergy;
return 0;
}

/*
```

```
// =====
// OPENMP REDUCTION FOR PARALLEL SUM
// =====
// #pragma omp parallel for schedule(static) reduction(+:totenergy)
// - Each thread maintains a PRIVATE copy of totenergy
// - Each thread sums a subset of grid points
// - At the end, all private copies are SUMMED together
//
// WHY REDUCTION?
// - Simple += would create DATA RACE (multiple threads writing same variable)
// - Reduction provides thread-safe way to compute aggregate values
// - Compiler handles synchronization automatically
//
```

#pragma omp – Serial implementation

OpenMP Code Optimization Impact: ofast vs ofast_omp_improved
(Identical compiler flags, different OpenMP pragmas, 1 energy source)



Test Parameters:

- Grid Size: 16384 x 16384
- Iterations: 500
- Platform: Leonardo DCGP (2x Intel Xeon Platinum 8480+, 112 cores) - 1 node used

The results demonstrate that the `-fopenmp` compiler flag acts solely as a runtime enabler. Without explicit `#pragma omp` directives, the `ofast` variant executes on a single master thread, leaving the remaining 111 cores idle.

Speedup fixed problem size:

$$S_n = \frac{T_1}{T_n}$$

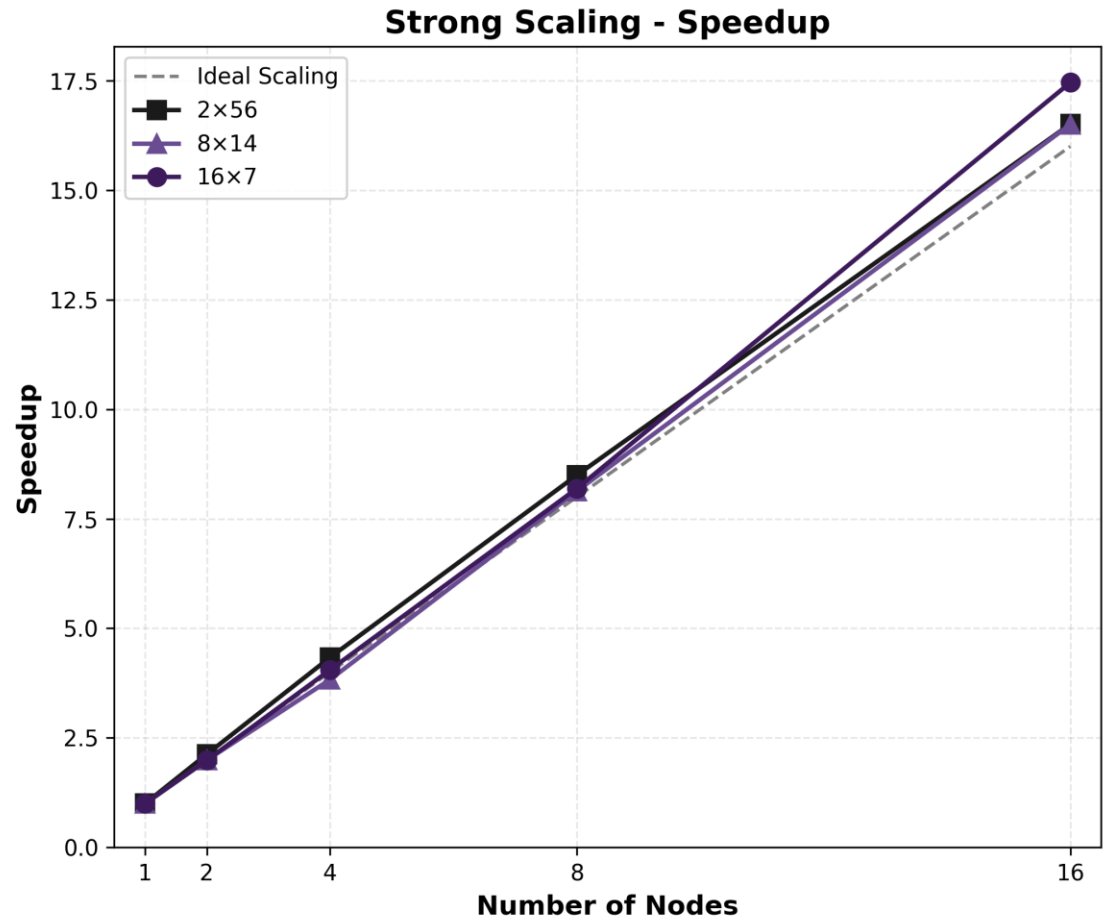
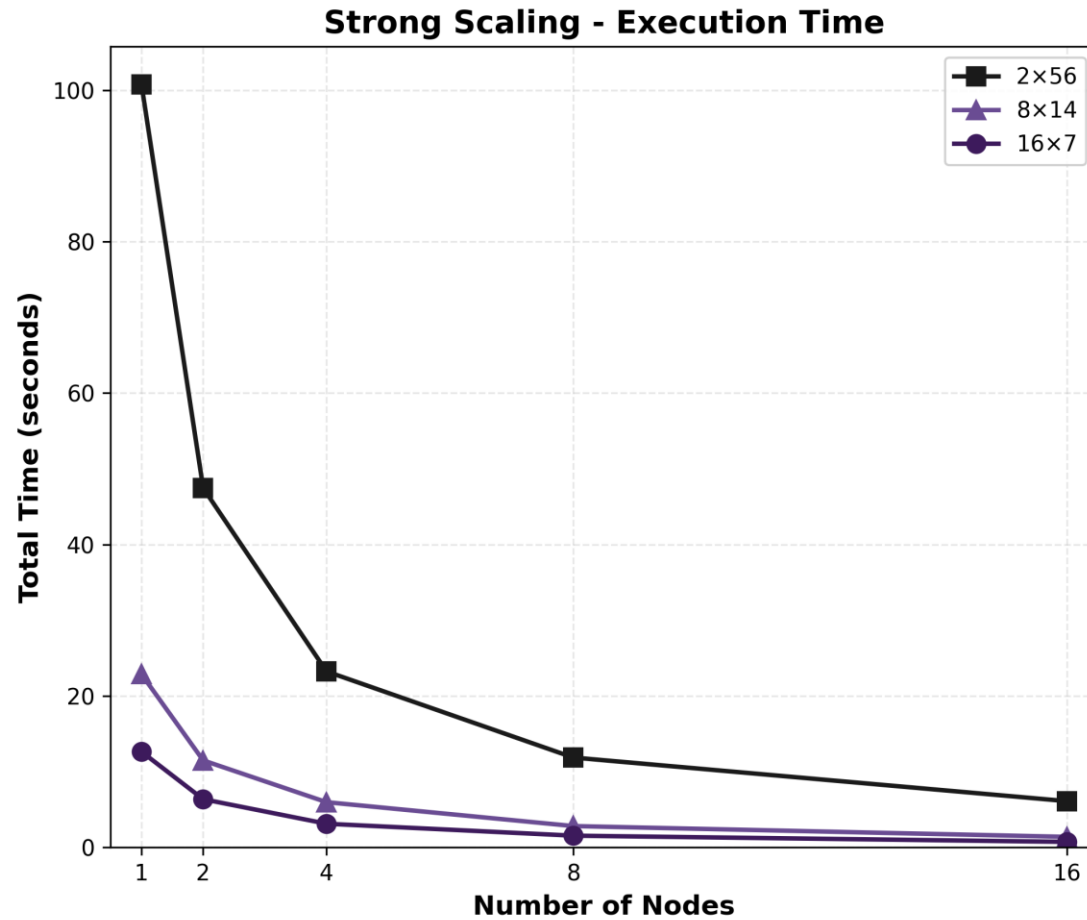
The dashed line represents Ideal Scaling. The divergence observed in `ofast_omp_improved` after 32 threads highlights the transition from a compute-limited regime to a memory-bandwidth limited regime.

OpenMP Scaling Efficiency:

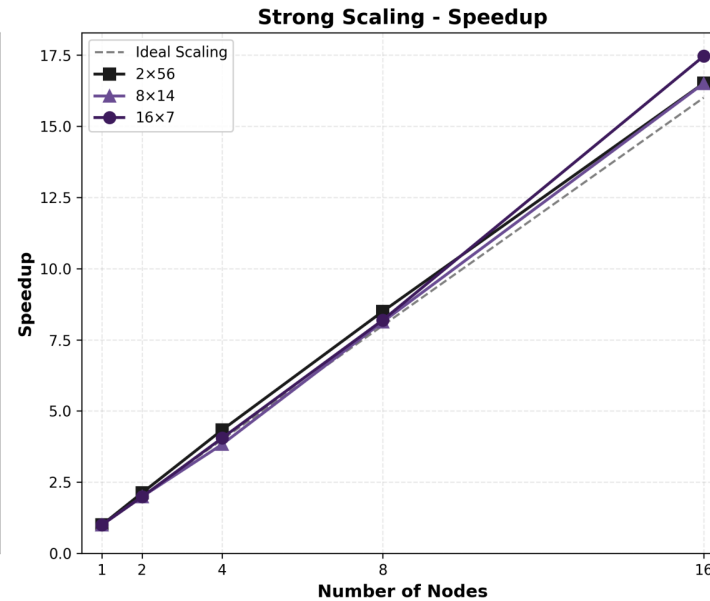
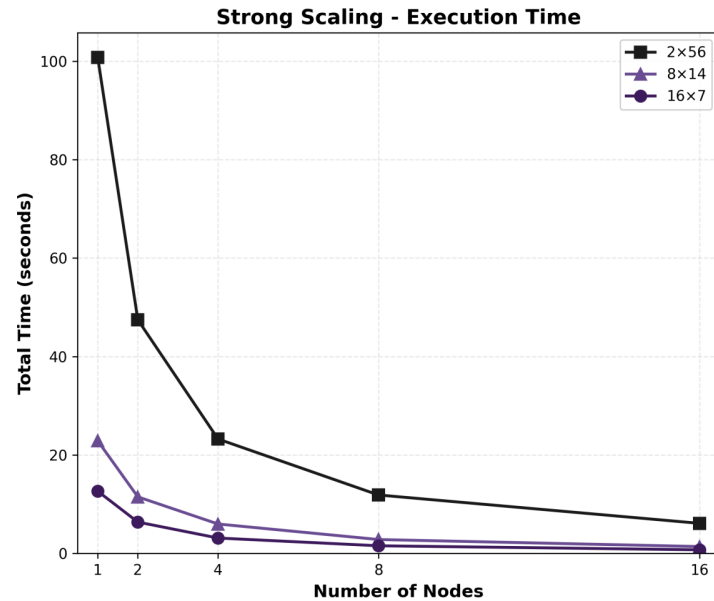
$$E_n = \frac{S_n}{n} \times 100\% = \frac{T_1}{n \cdot T_n} \times 100\%$$

Ideal behavior is represented by $E_n = 100\%$. The sharp decline in efficiency as threads increase is indicative of the memory wall. Since the 5-point stencil has low arithmetic intensity, memory bus becomes the primary bottleneck once the aggregate request rate from the cores exceeds the available bandwidth.

Strong Scaling Results (1/4)



Strong Scaling Results (2/4)



Test Parameters

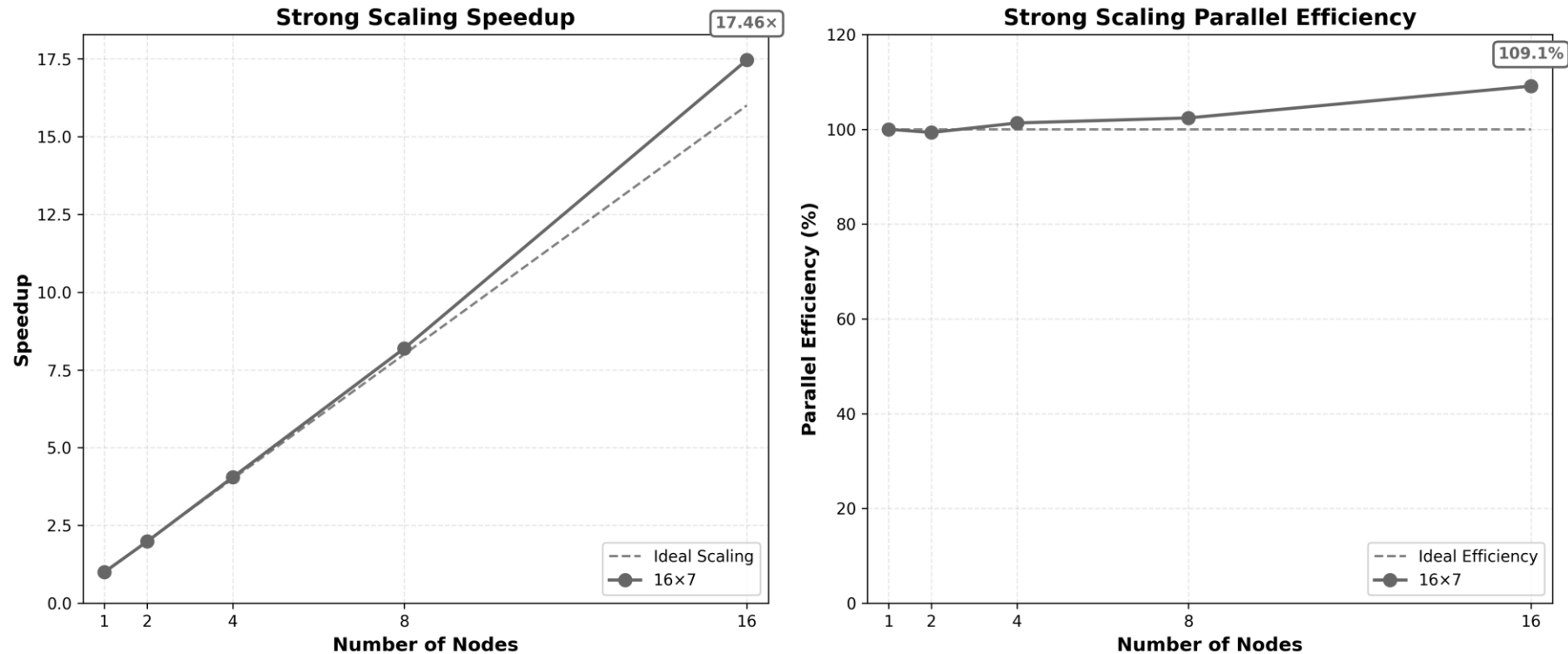
- **Grid Size:** 16384 x 16384
- **Iterations:** 500
- **Platform:** Leonardo DCGP – from 1 to 16 nodes
 - 16x7 (Many-MPI): 16 tasks/node, 7 threads/task
 - 8x14 (Standard): 8 tasks/node, 14 threads/task
 - 2x56 (Few-tasks): 2 tasks/node, 56 threads/task

Analysis of Results

- **2x56:** Too many threads sharing the same MPI rank create excessive contention for the memory bandwidth, resulting in a much slower baseline (100.77s vs 12.64s on 1 node).
- **8x14:** balanced configuration with 103.1% efficiency.
- **16x7:** achieved the fastest absolute time-to-solution (0.72s at 16 nodes) and the highest speedup (17.46x)

Config	Nodes	Time (s)	Speedup	Efficiency
2x56	16	6.10	16.51x	103.2 %
8x14	16	1.39	16.50x	103.1 %
16x7	16	0.72	17.46x	109.1 %

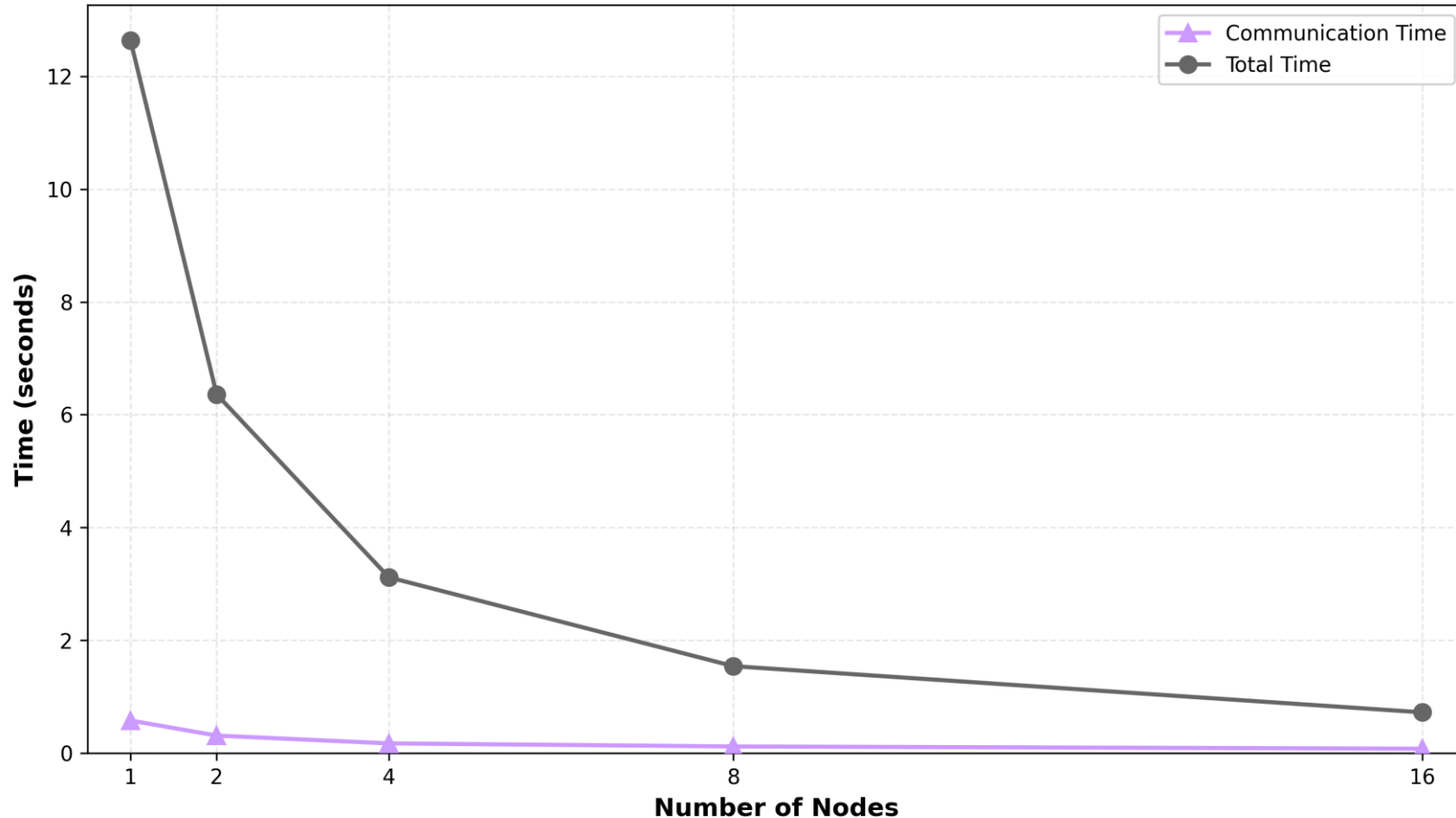
Strong Scaling Results (3/4)



- **At small scales**, the implementation follows the ideal line closely, achieving a speedup of 1.90x on 2 nodes (near-perfect efficiency)
- **At 16 nodes** (1792 cores), the implementation achieves a **17.46x** speedup with a parallel efficiency of **109.1%**:
- Performance surpasses the theoretical ideal (**Super-linear Scaling**), transcending standard expectations of Amdahl's Law; as the local problem size per node decreases, **cache fitting** allows the working set to reside in high-speed memory, effectively mitigating the memory-bandwidth bottleneck

Strong Scaling Results (4/4)

Strong Scaling - Execution Time (16×7 Configuration)

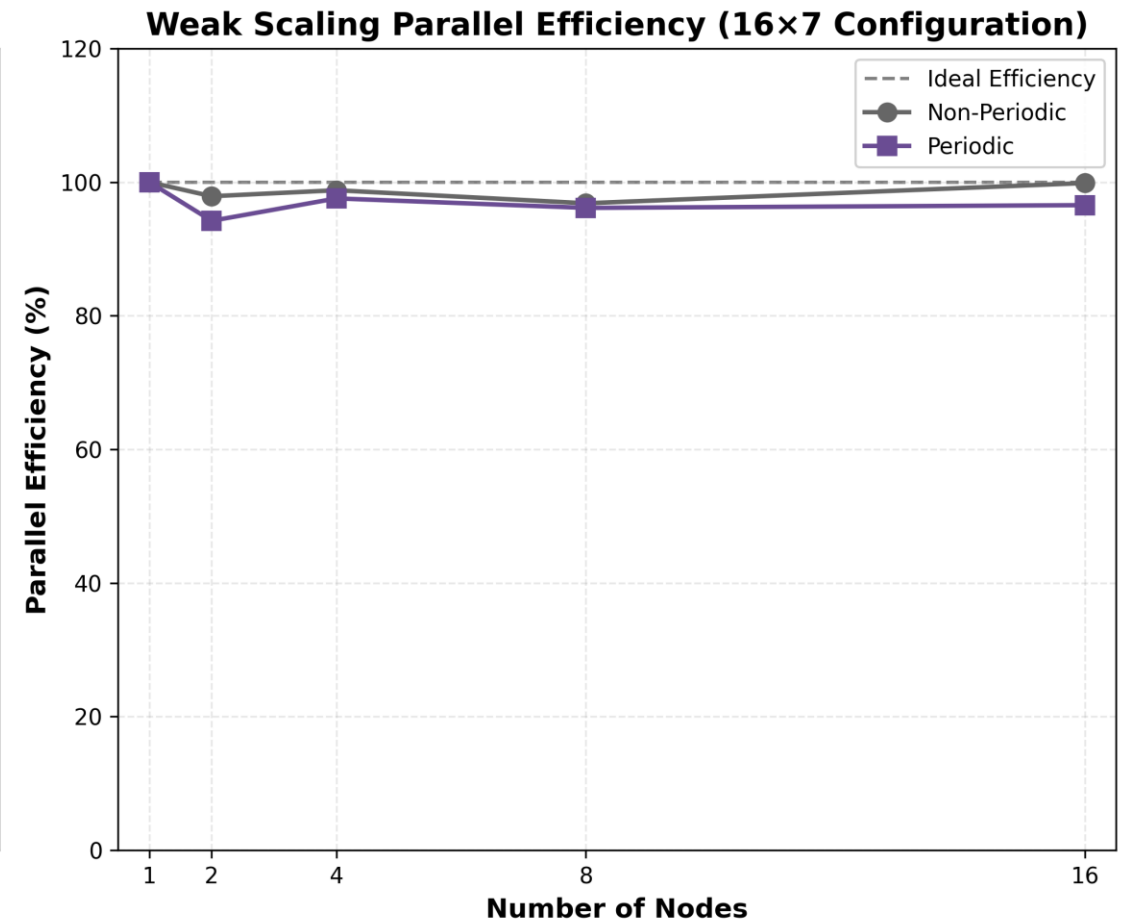
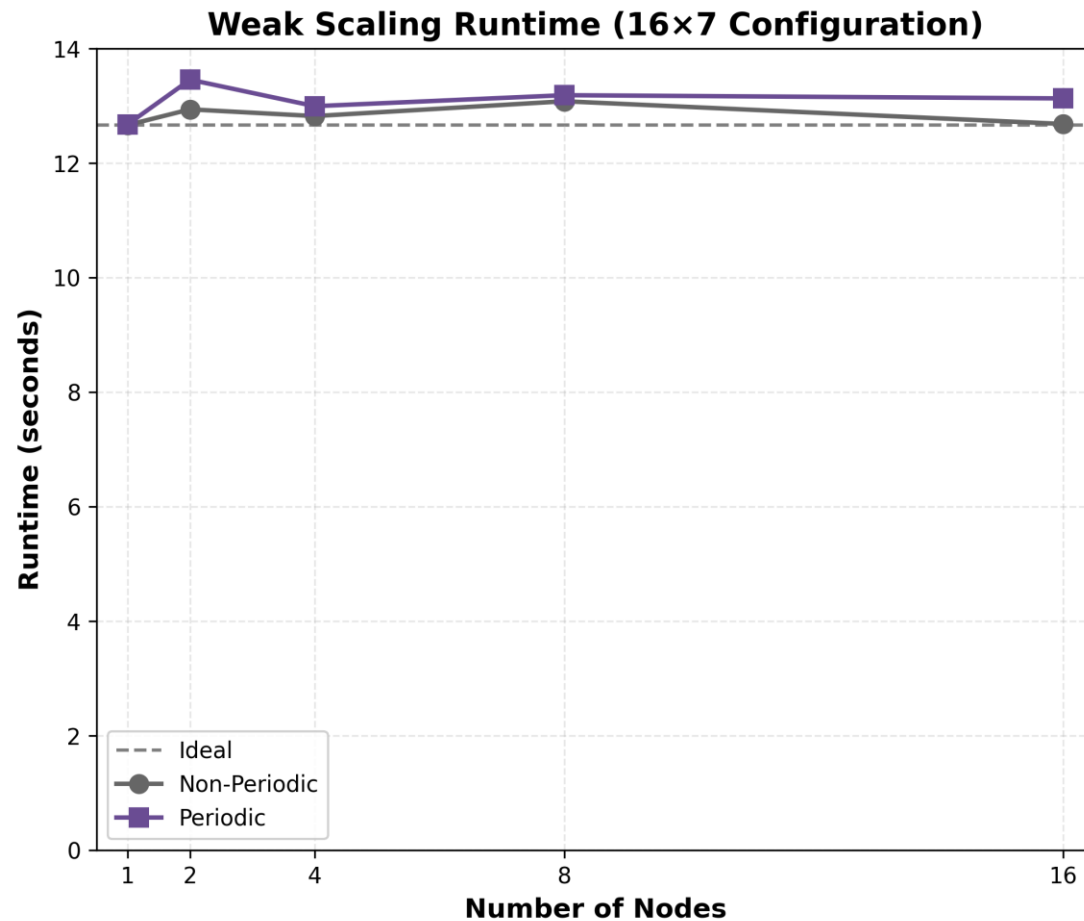


Low Communication Overhead

Despite scaling from 16 to 256 MPI tasks (1 to 16 nodes), the communication time represents a small and well-controlled fraction of total execution time:

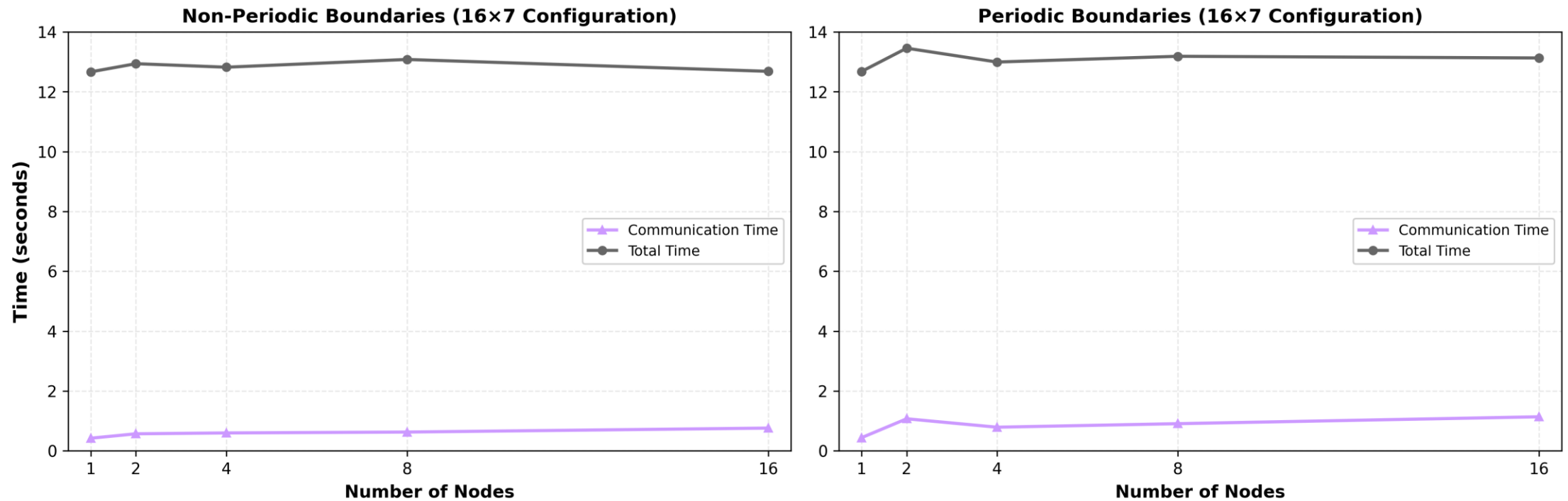
- **1 node:** 4.6% of 12.64s
- **2 nodes:** 4.9% of 6.36s
- **4 nodes:** 5.6% of 3.12s
- **8 nodes:** 7.7% of 1.54s
- **16 nodes:** 10.9% of 0.72s

Weak Scaling Results (1/3)



Weak Scaling Results (2/3)

Weak Scaling - Execution Time (as a function of nodes)



Weak Scaling Results (3/3)

Nodes (16x7)	Grid	Points (10 ⁹)	Runtime	Computation	Commu- nication	Efficiency	Runtime (Period.)	Computation (Period.)	Commu- nication (Period.)	Efficiency (Period.)
1	16384 ²	0.26	12.66	12.52	0.42	100.0 %	12.68	12.53	0.44	100.0 %
2	23170 ²	0.53	12.94	12.80	0.57	97.9 %	13.46	13.31	1.07	94.2 %
4	32768 ²	1.07	12.82	12.79	0.60	98.8 %	12.99	12.90	0.79	97.6 %
8	46340 ²	2.15	13.08	12.95	0.63	96.9 %	13.19	13.10	0.91	96.1 %
16	65536 ²	4.29	12.68	12.77	0.76	99.9 %	13.13	13.13	1.14	96.6 %

The implementation achieves excellent weak scaling (>96% efficiency) for both boundary conditions. The periodic boundary overhead is acceptable and expected, as wrap-around communication requires additional network routing compared to fixed boundaries.

Non-periodic boundaries maintain more consistent runtime across scales (coefficient of variation: 1.3%), while periodic boundaries show higher variability (2.6%), especially at 2 nodes where efficiency drops to 94.2%. This suggests that wrap-around communication introduces overhead that becomes more significant at smaller scales where boundary processes represent a larger fraction of total processes.

Communication time increases with problem size for both boundary types, but the absolute increase is modest (0.34s for non-periodic, 0.70s for periodic over 16× scaling), demonstrating that the communication-computation overlap strategy effectively manages the growing communication volume.

Main Achievements & Conclusions

- **Scalability:** The implementation achieves Super-linear Scaling (up to 109.1% efficiency) in strong scaling and near-perfect weak scaling (>96% efficiency).
- **Overlap:** The communication-computation overlap strategy (interior/border split) reduced communication overhead to <10% of total time.
- **NUMA:** The adoption of a NUMA-aware first-touch allocation strategy ensured data locality and prevented performance degradation from cross-socket memory access, effectively maximizing local bandwidth utilization.
- **OpenMP effect:** 18 pragmas led to a 20x speedup compared to non-parallelized code at 112 threads.
- **Memory Wall:** Efficiency drops beyond 16-32 threads within a node, confirming the stencil's nature as a memory-bandwidth bound application.

References

- Course Repository: <https://github.com/Foundations-of-HPC/High-Performance-Computing-2024>
- Project Repository: <https://github.com/egesia1/5-points-stencil>
- MPI Standard: <https://www.mpi-forum.org/docs/mpi-5.0/mpi50-report.pdf>
- OpenMP API: <https://www.openmp.org/>
- Leonardo HPC System: <https://leonardo-supercomputer.cineca.eu/it/leonardo-hpc-system/>

That's all folks,

"So long
and thanks
for all the fish"